

ບົດທີ 6: ການຮັບປະກັນຄຸນນະພາບ ແລະ ການທົດສອບຊອບແວ

Software Quality Assurance & Testing



ເນື້ອໃນຫຍໍ້

- 1 ພື້ນຖານຂອງຄຸນນະພາບ ແລະ ການທົດສອບຊອບແວ
- 2 ຄໍາສັບສໍາຄັນ: Bug, Error, Fault, Failure
- 3 ເປົ້າໝາຍຂອງການທົດສອບຊອບແວ (The Goal of Testing)
- 4 ການກວດສອບ (Verification) vs. ການຢືນຢັນ (Validation) - V&V
- 5 ລະດັບຂອງການທົດສອບ (Testing Levels)
- 6 ເທັກນິກການທົດສອບ
- 7 ແນວຄິດ Test-Driven Development (TDD)
- 8 ການທົດສອບແບບອັດຕະໂນມັດ (Automation Testing)

ແນວຄິດພື້ນຖານຂອງຄຸນນະພາບ ແລະ ການທົດສອບຊອບແວ

⚙️ ການປ່ຽນ Mindset

ຄຸນນະພາບຕ້ອງຖືກ **"ສ້າງຂຶ້ນ"**, ບໍ່ແມ່ນ **"ກວດສອບ"** ໃນຕອນທ້າຍ

⚠️ ຄຳສັບທີ່ສຳຄັນ

Error, Fault, Bug, Failure - ຄວາມແຕກຕ່າງລະຫວ່າງຄຳສັບເຫຼົ່ານີ້

💡 ເປົ້າໝາຍຂອງການທົດສອບ

ຄົ້ນຫາ Bug, ສ້າງຄວາມໝັ້ນໃຈ, ໃຫ້ຂໍ້ມູນສຳລັບການຕັດສິນໃຈ



ຄຳສັບສຳຄັນ: Bug, Error, Fault, Failure

⚙ Error (ຂໍ້ຜິດພາດ)

ເກີດຂຶ້ນໃນ **"ຫົວ"** ຂອງມະນຸດ
ການກະທຳທີ່ຜິດພາດຂອງໂປຣແກຣມເມີ



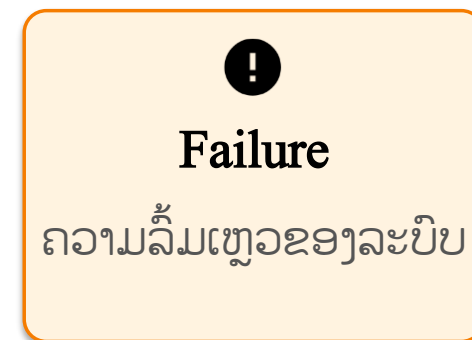
Fault (ຈຸດປົກພ່ອງ) ຫຼື Defect

ເປັນ **ຜິດພາດ** ຂອງ Error
"ຈຸດທີ່ຜິດພາດ" ທີ່ຖືກຂຽນເຂົ້າໄປໃນ Source Code



❗ Failure (ຄວາມຜິດປົກກະຕິ)

ເກີດຂຶ້ນເມື່ອ Fault ຖືກ **"ກະຕຸ້ນ"** ໃຫ້ເຮັດວຽກ
ພຶດຕິກຳຂອງຊອບແວ "ແຕກຕ່າງ" ໄປຈາກສິ່ງທີ່ຄາດຫວັງ



ຕົວຢ່າງ:

1. Error: ໂປຣແກຣມເມີເຂົ້າໃຈຜິດ, ຄິດວ່າການຫານດ້ວຍສູນຈະສົ່ງຄ່າ 0 ອອກມາ.
2. Fault/Bug: ລາວຂຽນໂຄດ $result = x / y$ ໂດຍບໍ່ມີການກວດສອບວ່າ y ເປັນສູນ ຫຼືບໍ່.
3. Failure: ເມື່ອຜູ້ໃຊ້ປ້ອນຄ່າ $y = 0$, ໂປຣແກຣມກໍ "crash" ຫຼື ສະແດງຂໍ້ຄວາມ "Division by zero error".

ເປົ້າໝາຍຂອງການທົດສອບຊອບແວ (The Goal of Testing)

❗ ຄວາມເຂົ້າໃຈທີ່ຜິດ

ຫຼາຍຄົນຄິດວ່າເປົ້າໝາຍຂອງການທົດສອບຄື
"ການພິສູດວ່າຊອບແວບໍ່ມີ Bug"

"Testing can only show the presence of bugs, not their absence."

(ການທົດສອບສາມາດສະແດງໃຫ້ເຫັນເຖິງ "ການມີຢູ່" ຂອງ Bug, ບໍ່ແມ່ນ "ການບໍ່ມີຢູ່" ຂອງມັນ)



ຄົ້ນຫາ Bug



ສ້າງຄວາມໝັ້ນໃຈ



ໃຫ້ຂໍ້ມູນ

ການກວດສອບ (Verification) vs. ການຢືນຢັນ (Validation) - V&V

✓ Verification

"Are we building the product right?"

- 📄 ກວດສອບກັບ ເອກະສານການອອກແບບ ແລະ ເອກະສານຄວາມຕ້ອງການ
- 🔧 ກິດຈະກຳ: Code Reviews, Walkthroughs, Inspections
- 📌 ການກວດສອບ "ພາຍໃນ" ຂອງລະບົບ



VS

✓ Validation

"Are we building the right product?"

- 👤 ກວດສອບວ່າຕອບໂຈດ ຄວາມຕ້ອງການທີ່ແທ້ຈິງຂອງຜູ້ໃຊ້
- 📌 ກິດຈະກຳ: ການທົດສອບທຸກລະດັບ, ໂດຍສະເພາະ User Acceptance Testing (UAT)
- 👁️ ການກວດສອບ "ພາຍນອກ" ຈາກມຸມມອງຂອງຜູ້ໃຊ້



ລະດັບຂອງການທົດສອບ (Testing Levels)

◆ 4 ລະດັບຫຼັກຂອງການທົດສອບ

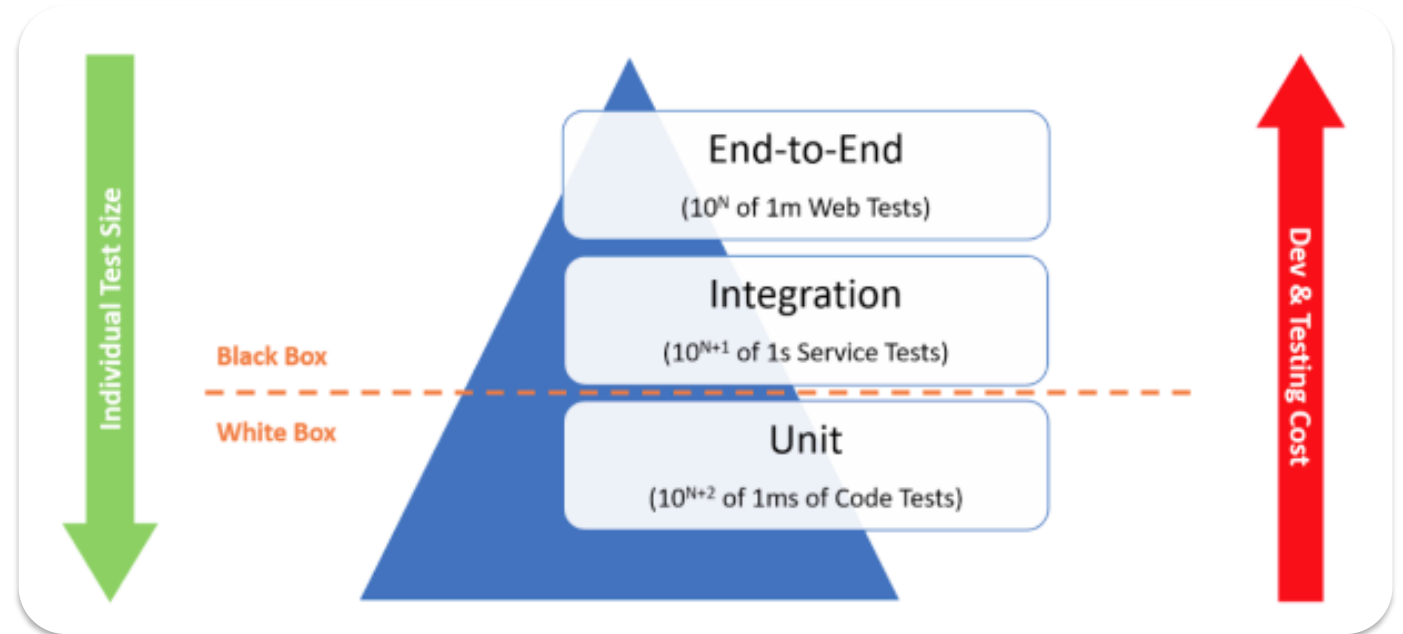
ໃນການພັດທະນາຊອບແວ, ເຮົາຈະແບ່ງ
ການທົດສອບອອກເປັນ 4 ລະດັບຫຼັກ

⤴ Test Pyramid

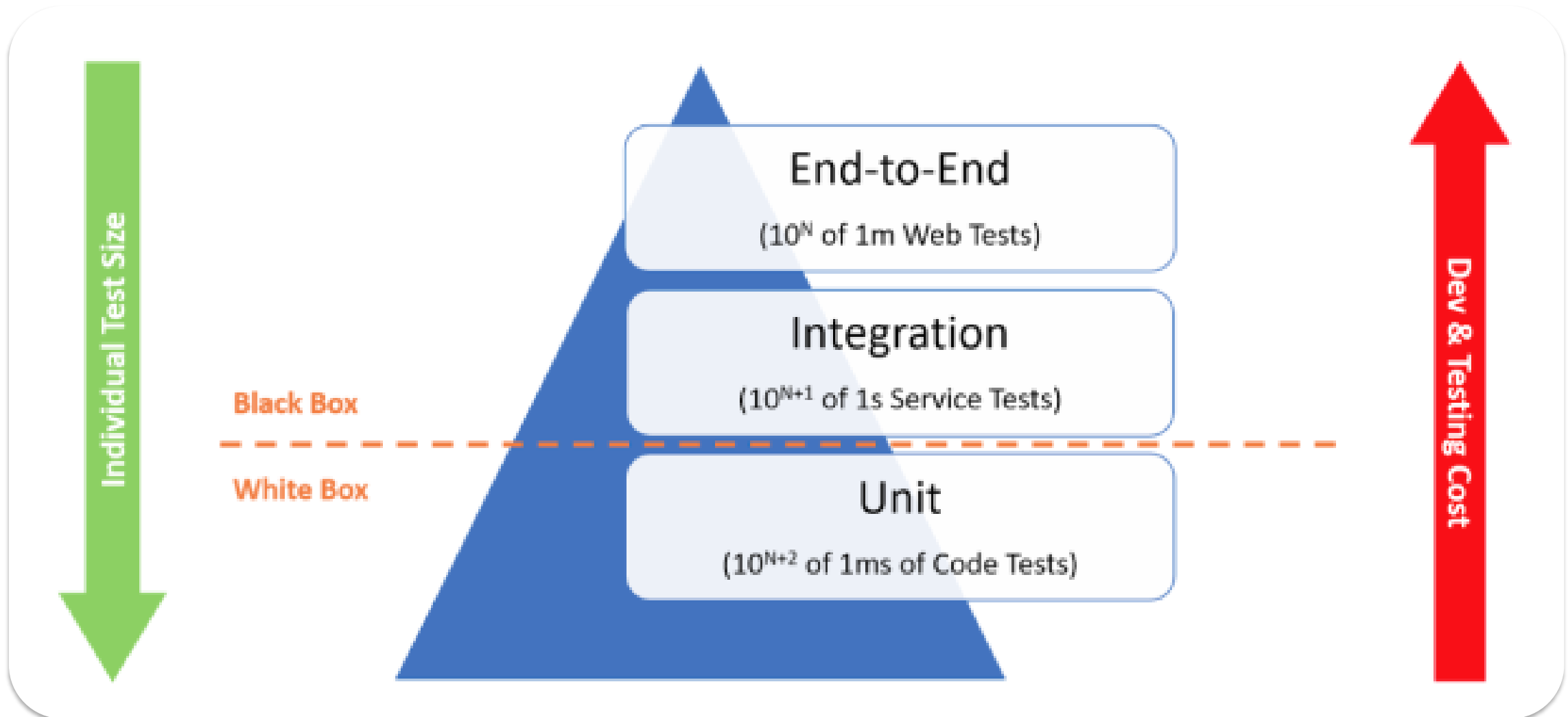
ແນວຄິດນີ້ມັກຈະຖືກສະແດງດ້ວຍພາບທີ່
ເອີ້ນວ່າ **"Test Pyramid"**



ການປຽບທຽບ: ໃຫ້ຄິດເຖິງການ
"ປະກອບລົດຍົນ"



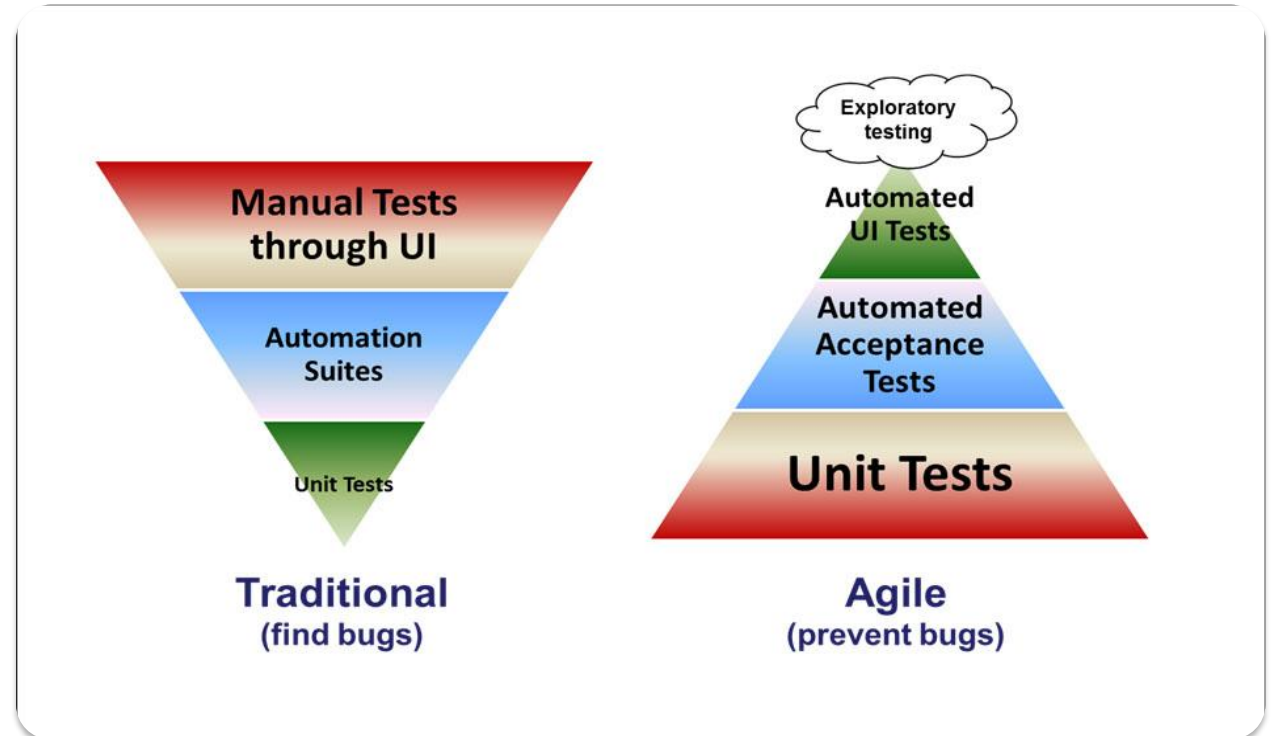
ລະດັບຂອງການທົດສອບ (Testing Levels)



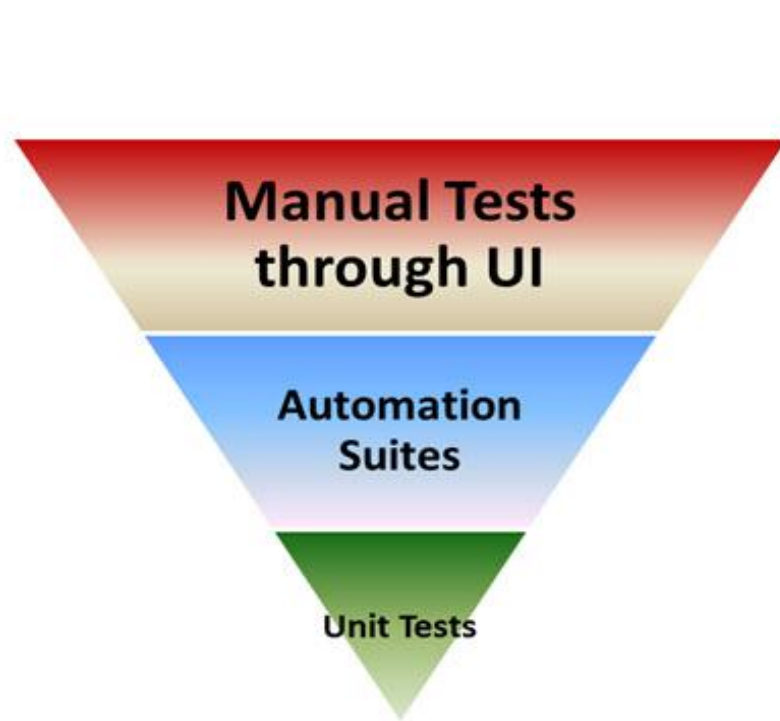
Unit Testing (ການທົດສອບລະດັບໜ່ວຍຍ່ອຍ)

🔑 ການທົດສອບໜ່ວຍທີ່ນ້ອຍທີ່ສຸດ

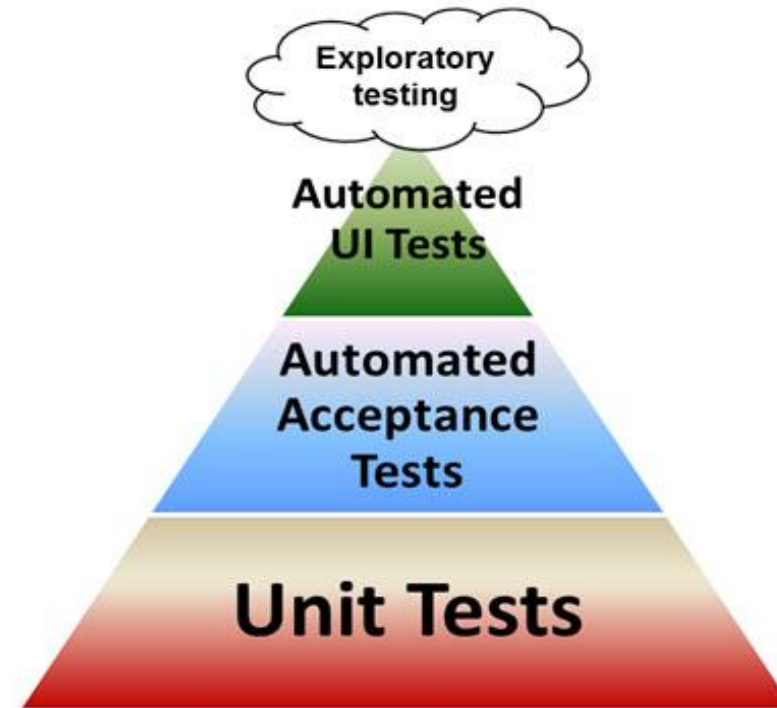
- ❓ **ຄໍາຖາມ:** ສ່ວນປະກອບແຕ່ລະອັນ ເຮັດວຽກຖືກຕ້ອງຕາມໜ້າທີ່ຂອງມັນ ຫຼືບໍ່?
- 📖 **ຄໍາອະທິບາຍ:** ຄືການທົດສອບ "ໜ່ວຍທີ່ນ້ອຍທີ່ສຸດ" ຂອງໂຄດ
- 👤 **ໃຜເປັນຄົນເຮັດ:** ນັກພັດທະນາ (Developer)
- 🕒 **ເຮັດເມື່ອໃດ:** ເຮັດຕະຫຼອດເວລາໃນຂະນະທີ່ກຳລັງຂຽນໂຄດ
- 🏠 **ການປຽບທຽບ:** ຄືການກວດສອບວ່າ "ນ້ອດ" ແຕ່ລະໂຕມີຂະໜາດຖືກຕ້ອງ



Unit Testing (ການທົດສອບລະດັບໜ່ວຍຍ່ອຍ)



Traditional
(find bugs)



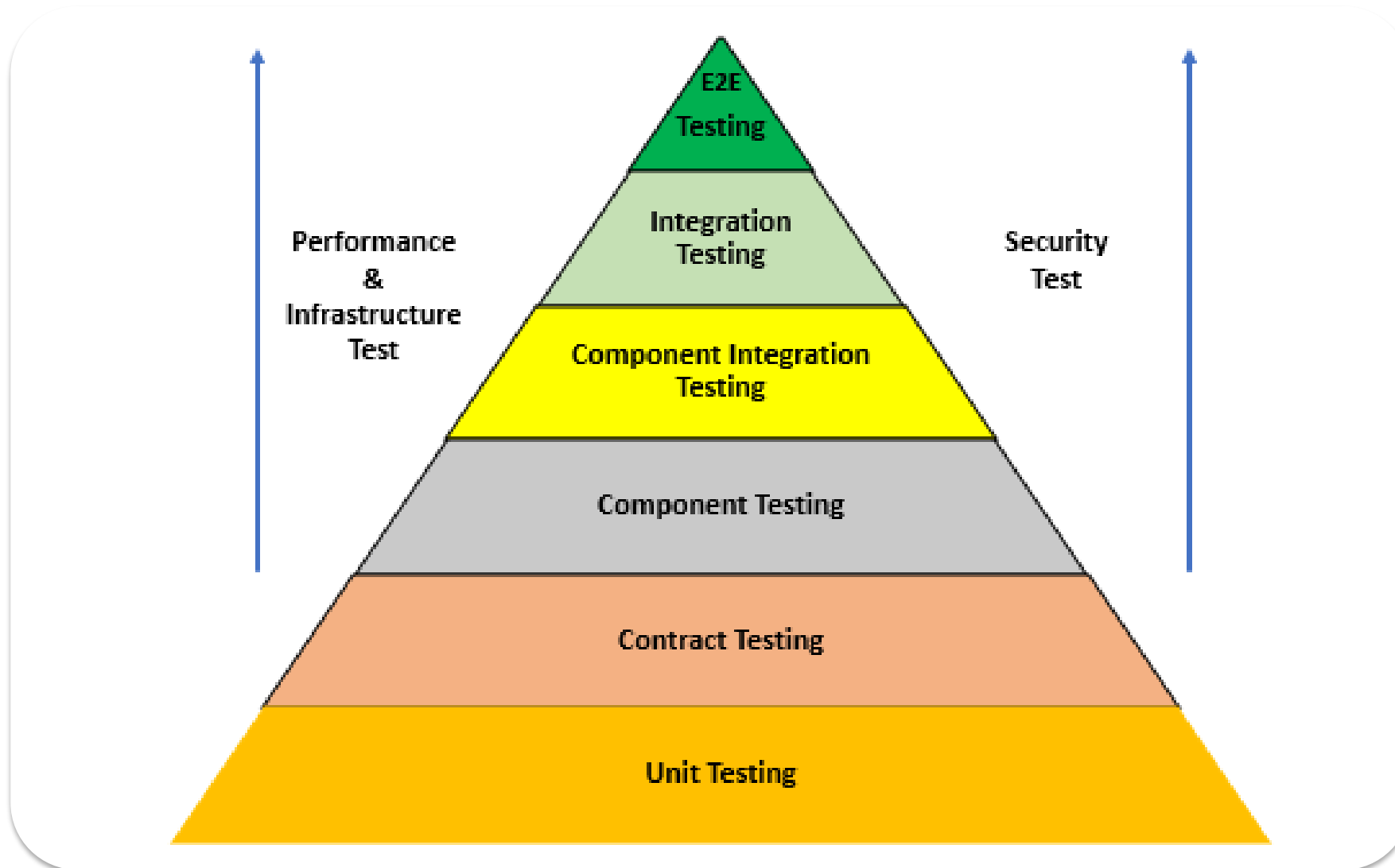
Agile
(prevent bugs)

Integration Testing (ການທົດສອບການເຊື່ອມໂຍງ)

🔗 ການທົດສອບການເຊື່ອມໂຍງ

- ① **ຄໍາຖາມ:** ເມື່ອນຳເອົາສ່ວນປະກອບຕ່າງໆ ມາປະກອບເຂົ້າກັນ, ເຮັດວຽກຮ່ວມກັນໄດ້ຖືກຕ້ອງ ຫຼືບໍ່?
- 📖 **ຄໍາອະທິບາຍ:** ຄືການທົດສອບ **"ການໂຕ້ຕອບ"** ລະຫວ່າງ Unit ຫຼື Module ຕ່າງໆ
- 👤 **ໃຜເປັນຄົນເຮັດ:** ນັກພັດທະນາ (Developer) ຫຼື ທີມ QA
- 🕒 **ເຮັດເມື່ອໃດ:** ຫຼັງຈາກທີ່ Unit Test ຜ່ານ
- 🔧 **ການປຽບທຽບ:** ຄືການນຳເອົາ **"ເຄື່ອງຈັກ"** ມາເຊື່ອມຕໍ່ກັບ **"ລະບົບເກຍ"**

Integration Testing (ການທົດສອບການເຊື່ອມໂຍງ)



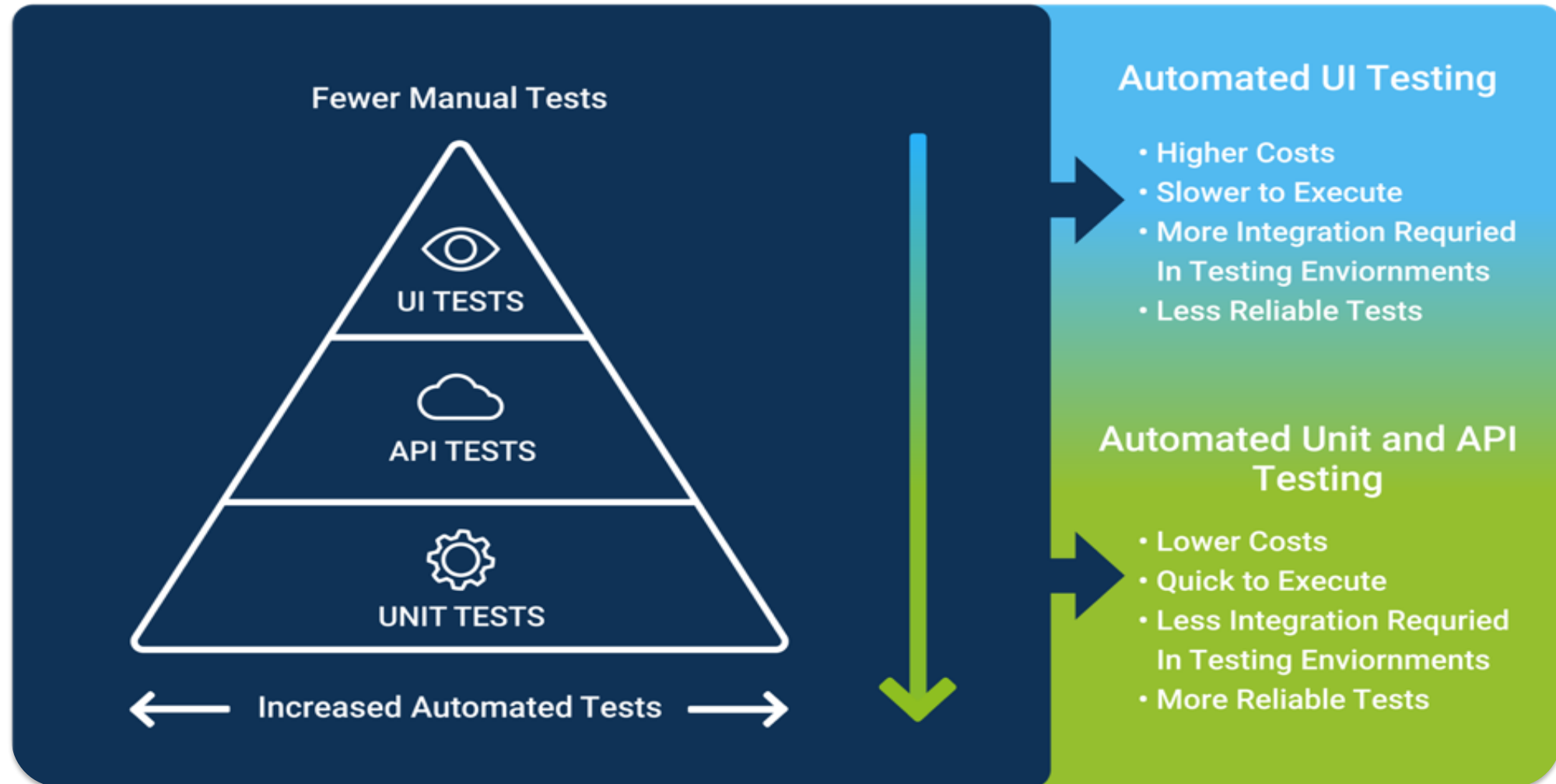
System Testing (ການທົດສອບລະດັບລະບົບ)

໔໓ ການທົດສອບລະບົບທີ່ສົມບູນ

- ໑ ຄໍາຖາມ: ລະບົບທັງໝົດທີ່ປະກອບສົມບູນແລ້ວ ເຮັດວຽກໄດ້ຖືກຕ້ອງຕາມ "ຄວາມຕ້ອງການ" ທັງໝົດ ຫຼືບໍ່?
- ໒ ຄໍາອະທິບາຍ: ຄືການທົດສອບ "ລະບົບທີ່ສົມບູນ" ໃນສະພາບແວດລ້ອມທີ່ໃກ້ຄຽງກັບສະພາບແວດລ້ອມຈິງທີ່ສຸດ
- ໓ ໃຜເປັນຄົນເຮັດ: ທີມ QA ເປັນຫຼັກ
- ໔ ເຮັດເມື່ອໃດ: ຫຼັງຈາກທີ່ Integration Testing ຜ່ານ
- ໕ ການປຽບທຽບ: ຄືການນຳເອົາ "ລົດທັງຄັນ" ອອກໄປຂັບທົດສອບເທິງຖະໜົນຈິງ

System Testing (ການທົດສອບລະດັບລະບົບ)

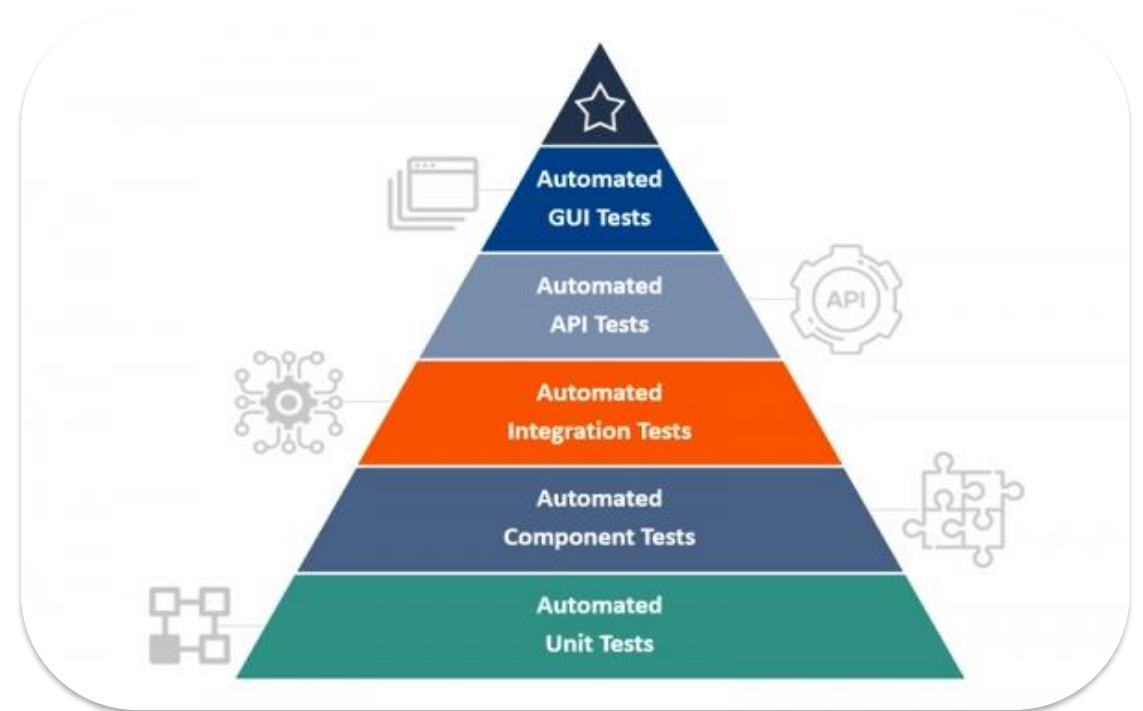
The Automation Pyramid



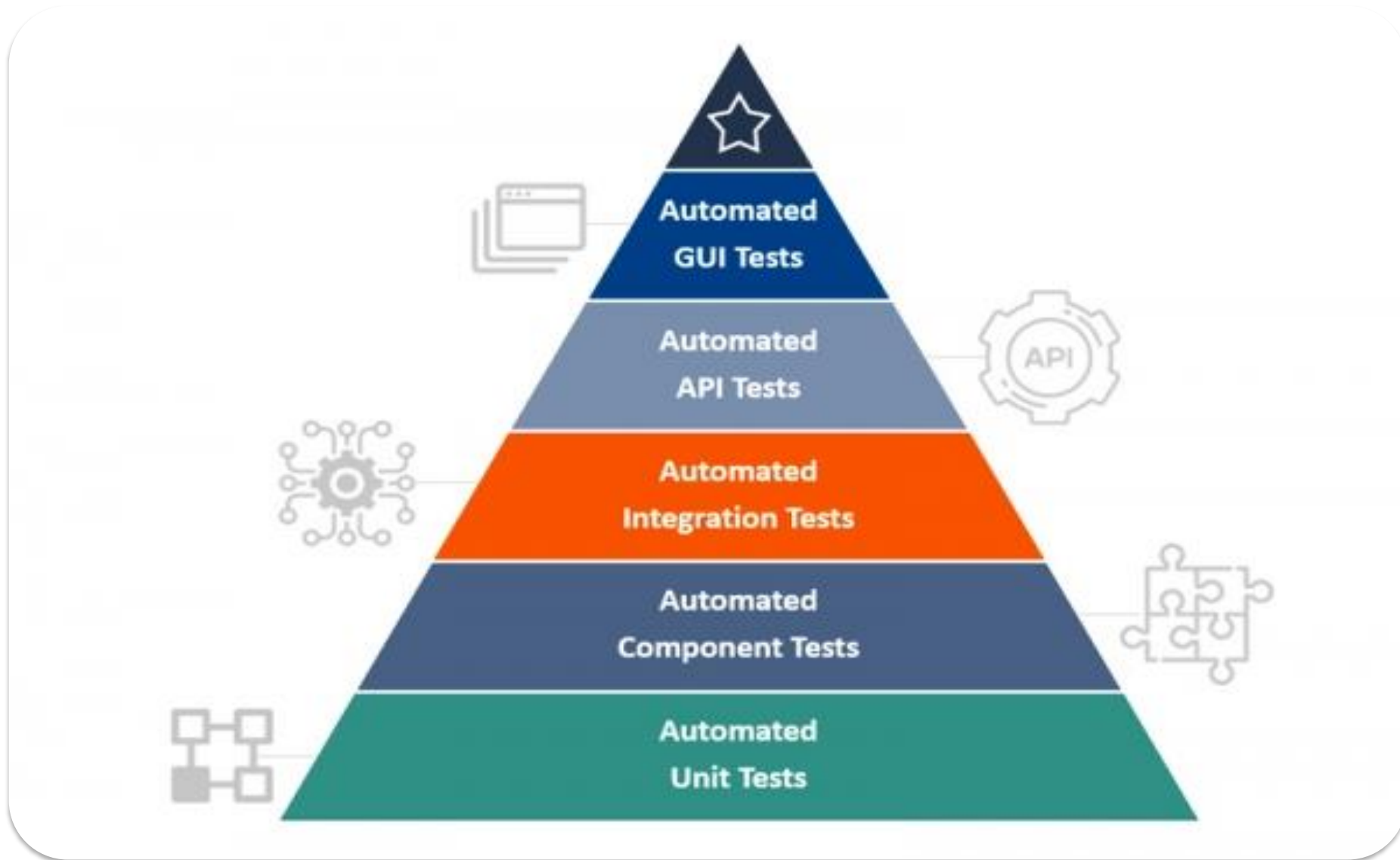
User Acceptance Testing (UAT) (ການທົດສອບການຍອມຮັບໂດຍຜູ້ໃຊ້)

✓ ການທົດສອບການຍອມຮັບໂດຍຜູ້ໃຊ້

- ① **ຄໍາຖາມ:** ລະບົບທີ່ສ້າງຂຶ້ນນັ້ນ, ມັນແມ່ນ **"ສິ່ງທີ່ຜູ້ໃຊ້ຕ້ອງການ"** ແລະ ສາມາດນໍາໄປໃຊ້ແກ້ໄຂບັນຫາ
- 📖 **ຄໍາອະທິບາຍ:** ນີ້ຄືການທົດສອບ **"ຢືນຢັນ"** ວ່າຊອບແວຕອບໂຈດຄວາມຕ້ອງການທາງທຸລະກິດ
- 👤 **ໃຜເປັນຄົນເຮັດ:** ລູກຄ້າ (Customer) ຫຼື ຜູ້ໃຊ້ຕົວຈິງ (End-users)
- 🕒 **ເຮັດເມື່ອໃດ:** ເປັນຂັ້ນຕອນສຸດທ້າຍ ຫຼັງຈາກທີ່ System Testing ຜ່ານ
- 🏢 **ການປຽບທຽບ:** ຄືການໃຫ້ **"ເຈົ້າຂອງລົດ"** ມາລອງຂັບລົດຄັນໃໝ່ຂອງຕົນເອງ



User Acceptance Testing (UAT) (ການທົດສອບການຍອມຮັບໂດຍຜູ້ໃຊ້)



ເທັກນິກການທົດສອບ: Black-box Testing

📁 ການທົດສອບແບບກ່ອງດຳ

- 🔗 **ແນວຄິດຫຼັກ:** ຜູ້ທົດສອບເບິ່ງລະບົບເປັນ "ກ່ອງສີດຳ"
- 🔍 **ຈຸດສຸມ:** ທົດສອບໂດຍອີງໃສ່ "ຄວາມຕ້ອງການ" ແລະ "ພຶດຕິກຳພາຍນອກ"
- ⑦ **ຄຳຖາມຫຼັກ:** ເມື່ອເຮົາໃສ່ "Input" ແລະ "Output" ຖືກຕ້ອງບໍ?
- 🔧 **ການປຽບທຽບ:** ຄືກັບການທົດສອບ "ເຕົາໄມໂຄເວຟ"

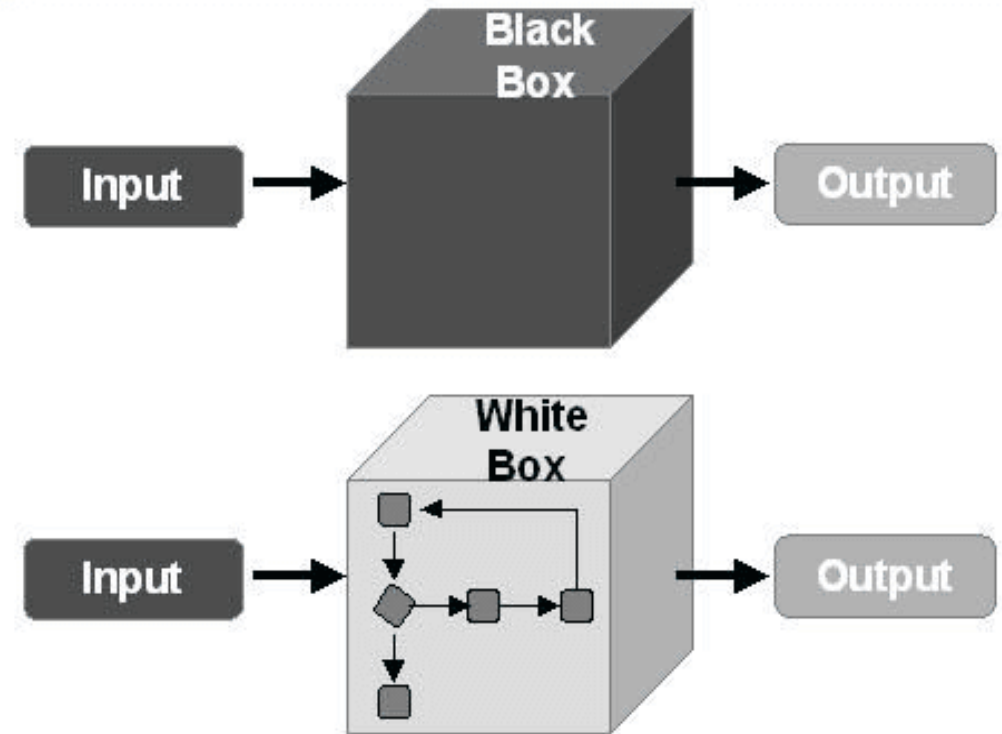


Equivalence Partitioning



Boundary Value Analysis

Comparison among Black-Box & White-Box Tests



www.softwaretestinggenius.com

ເທັກນິກການທົດສອບ: White-box Testing

🕒 ການທົດສອບແບບກ່ອງຂາວ

- 🕒 **ແນວຄິດຫຼັກ:** ຜູ້ທົດສອບເບິ່ງລະບົບເປັນ "ກ່ອງໃສ"
- 🔧 **ຈຸດສຸມ:** ທົດສອບໂດຍອີງໃສ່ "ເສັ້ນທາງການເຮັດວຽກ" ຂອງໂຄດ
- ① **ຄໍາຖາມຫຼັກ:** Test Case ສາມາດເຮັດໃຫ້ໂຄດທຸກແຖວ ຫຼື ທຸກເງື່ອນໄຂ ເຮັດວຽກຄົບຖ້ວນແລ້ວ ຫຼືບໍ່?
- 🔧 **ການປຽບທຽບ:** ຄືກັບ "ຊ່າງສ້ອມແປງໂທລະພາບ"

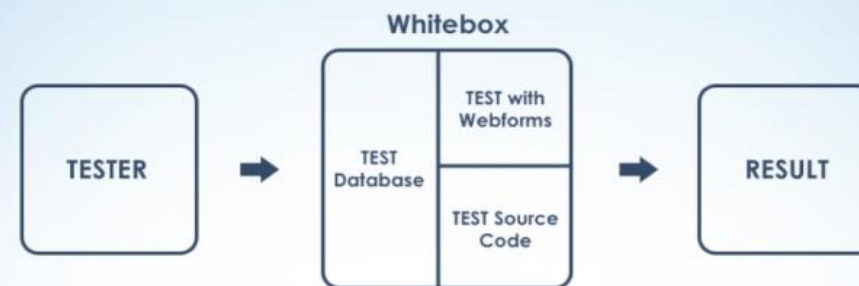
<>

Statement Coverage

↕

Branch Coverage

WHITE BOX TESTING APPROACH



ເທັກນິກການທົດສອບ: White-box Testing

WHITE BOX TESTING APPROACH



ເທັກນິກການທົດສອບ: ສະຫຼຸບ

↔ ການປະສົມປະສານເທັກນິກທັງສອງ

✓ ຄໍາຕອບຄື: ຕ້ອງໃຊ້ທັງສອງຢ່າງ!

🔗 Black-box ແລະ White-box ເປັນ "ເຄື່ອງມືທີ່ສົ່ງເສີມຊຶ່ງກັນ"

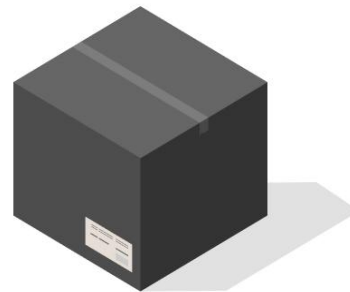
🔧 **White-box:** ການຮັບປະກັນຄຸນນະພາບ "ພາຍໃນ" (Developer)

👤 **Black-box:** ການຮັບປະກັນຄຸນນະພາບ "ພາຍນອກ" (QA)



ຍຸດທະສາດທີ່ດີ: ປະສົມປະສານທັງສອງ
ເທັກນິກໃນ **Test Pyramid**

QA testers



Black box - we do not
know anything

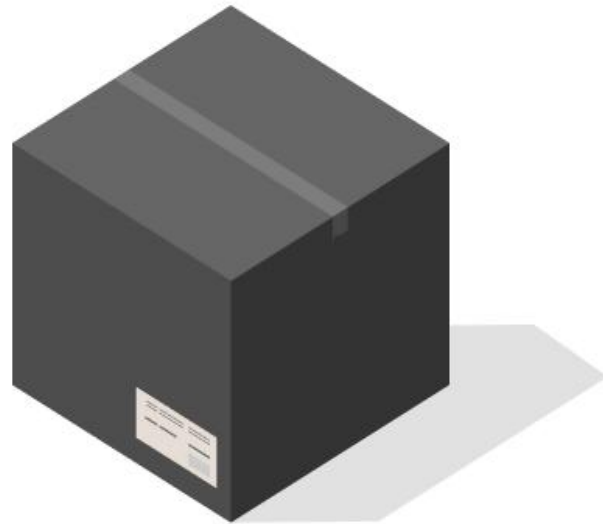
Developers



White box - we know
everything

ເທັກນິກການທົດສອບ: ສະຫຼຸບ

QA testers



Black box - we do not
know anything

Developers



White box - we know
everything

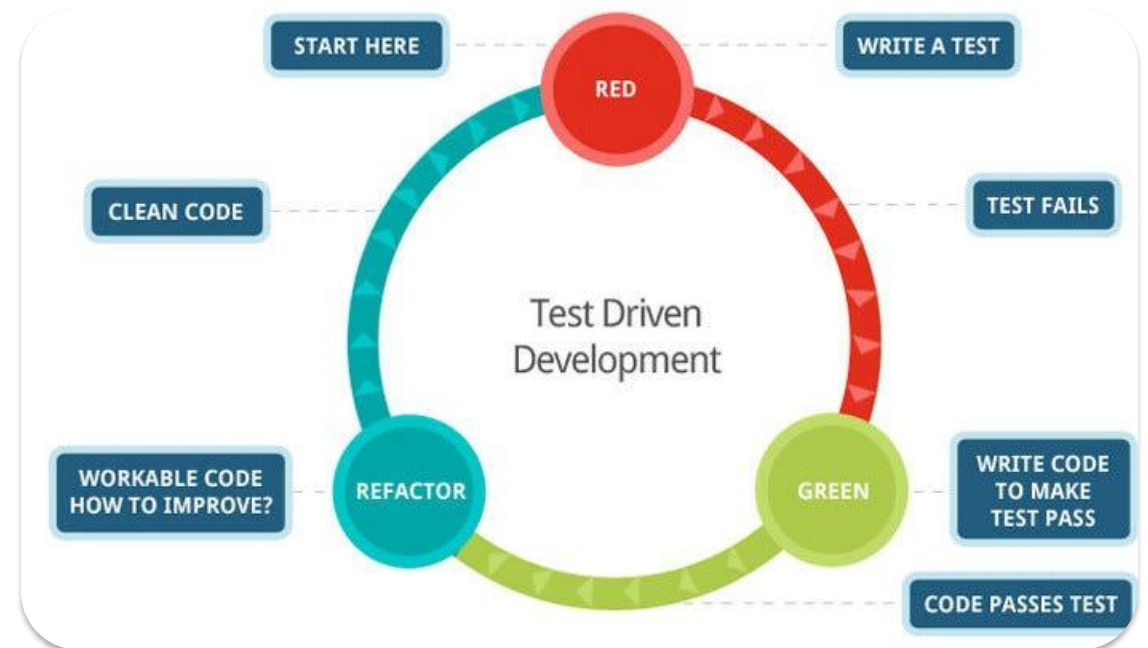
ແນວຄິດ Test-Driven Development (TDD)

↻ ການຂຽນ Test ກ່ອນຂຽນໂຄດ

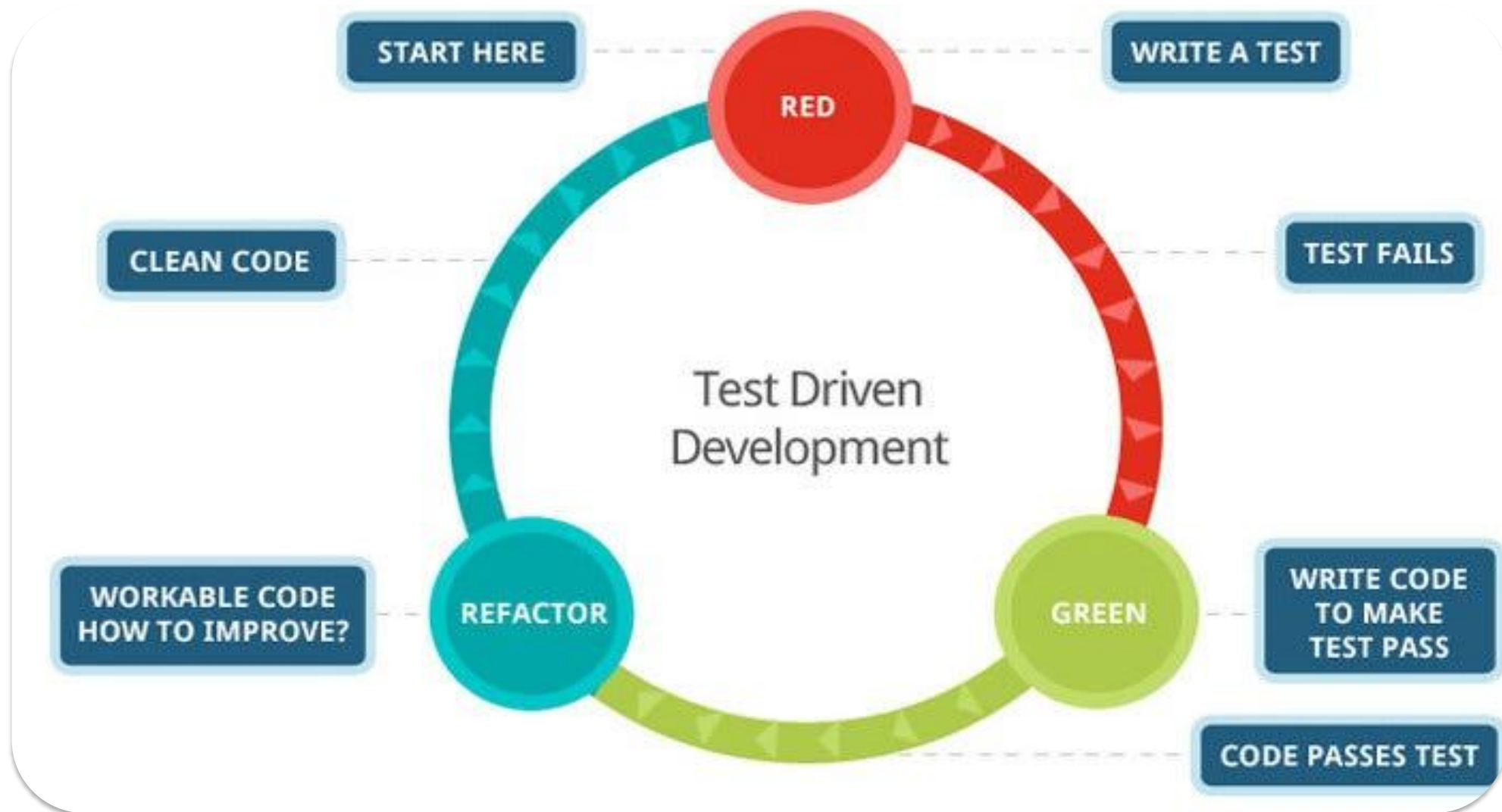
- ① **TDD ແມ່ນຫຍັງ?** ຄືຂະບວນການພັດທະນາຊອບແວ ທີ່ເຮົາຈະ **"ຂຽນ Test ກ່ອນທີ່ຈະຂຽນໂຄດ"**
- ② **ວົງຈອນ TDD: RED** - ຂຽນ Test ທີ່ລົ້ມ, **GREEN** - ຂຽນໂຄດໃຫ້ຜ່ານ, **REFACTOR** - ປັບປຸງໂຄດ

🔗 ວົງຈອນ **Red-Green-Refactor** ເປັນຫົວໃຈຂອງ TDD

- | | |
|------------------|--------------------|
| 🛡️ Safety Net | 👤 ປັບປຸງການອອກແບບ |
| 📄 ເອກະສານມີຊີວິດ | 🕒 ຫຼຸດເວລາໃນ Debug |



ແນວຄິດ Test-Driven Development (TDD)



ການທົດສອບແບບອັດຕະໂນມັດ (Automation Testing)

🏠 ການໃຊ້ "ຊອບແວ" ເພື່ອທົດສອບ "ຊອບແວ"

① ການທົດສອບແບບອັດຕະໂນມັດ: ໃຊ້ Test Scripts ເພື່ອທົດສອບອີກຂຕໜຶ່ງ

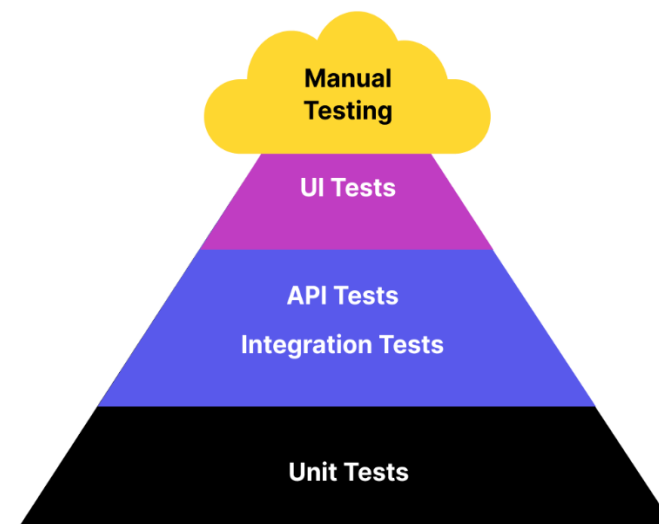
🔍 ຄວາມໄວ - Test Scripts ເຮັດວຽກໄວກວ່າມະນຸດຫຼາຍຮ້ອຍ

🔄 ການເຮັດຊໍ້າໄດ້ - ລັນ Test ຊຸດດຽວກັນໄດ້ເລື້ອຍໆ

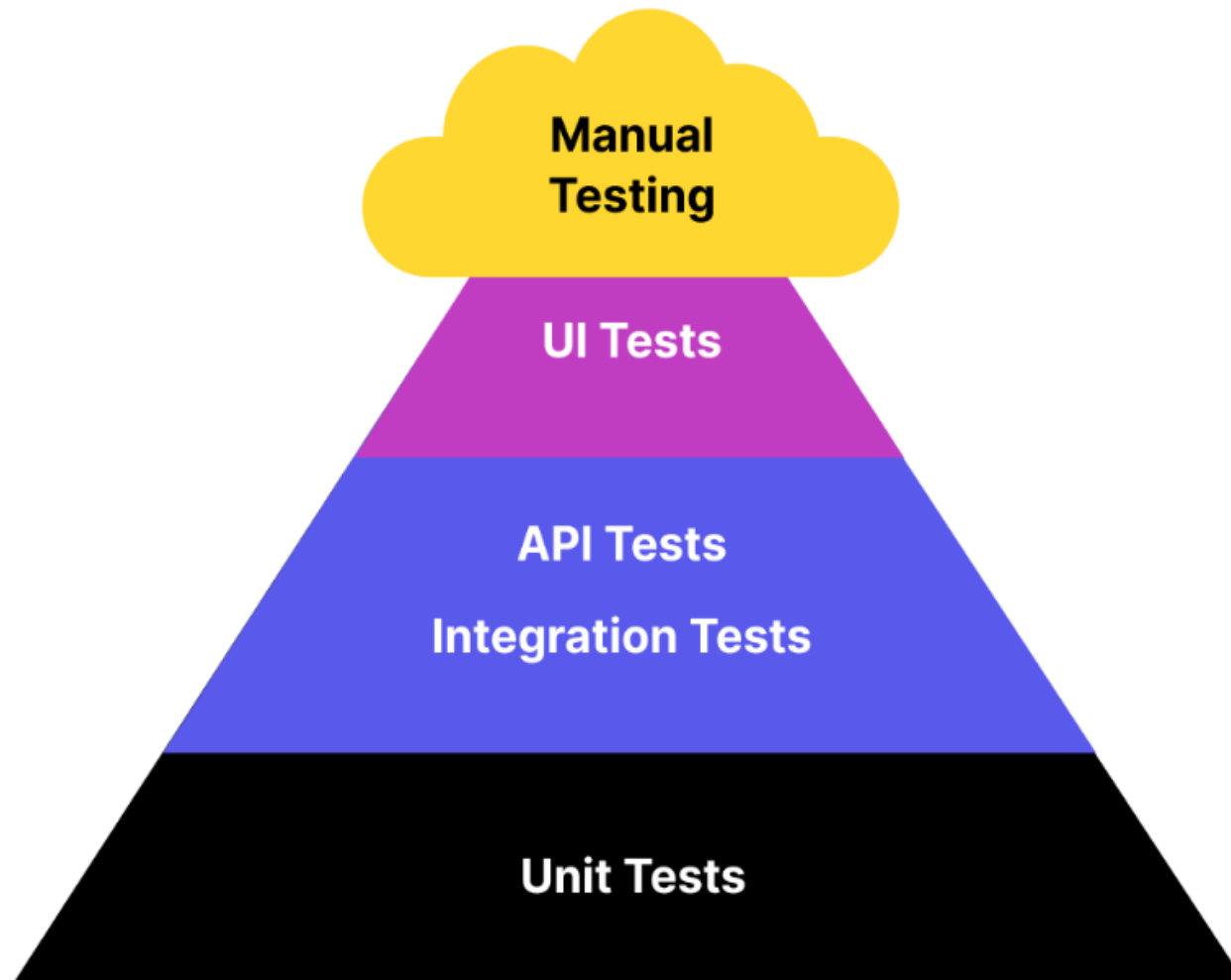
🛡️ ຄວາມໜ້າເຊື່ອຖື - ບໍ່ມີອາການເບື້ອ, ເມື່ຍ, ຫຼື ມອງຂ້າມລາຍ



ສະໜັບສະໜູນ **CI/CD Pipeline** ໄດ້



ການທົດສອບແບບອັດຕະໂນມັດ (Automation Testing)



ບົດສະຫຼຸບ

📄 ຈຸດສຸມຂອງການຮັບປະກັນຄຸນນະພາບຊອບແວ

🧠 **Mindset ທີ່ຖືກຕ້ອງ:** ຄຸນນະພາບເປັນ "ຄວາມຮັບຜິດຊອບຂອງທຸກຄົນ"

🔹 **4 ລະດັບການທົດສອບ:** Unit, Integration, System, UAT

➔ **ເທັກນິກທັງສອງ:** Black-box ແລະ White-box

📋 **ການພັດທະນາທີ່ທັນສຸດ:** TDD ແລະ ການທົດສອບອັດຕະໂນມັດ

✅ ສ້າງຕາໜ່າງຄວາມປອດໄພ 🛡️ ຮອບດ້ານທີ່ສຸດ

🔄 ການປັບປຸງຕົວເລື້ອຍ 🤖 ການທົດສອບອັດຕະໂນມັດ

